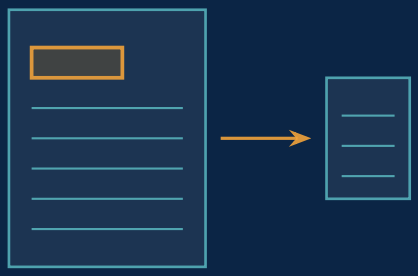
ICT UNIVERSITY · FACULTY OF ICT · DEPARTMENT OF COMPUTER SCIENCE



# DocuMind AI

## Intelligent Document Processing and Automated ERP Ingestion System

Project Requirements, Architecture, UML Models and Implementation Plan

**OCR · INFORMATION EXTRACTION · VALIDATION · ERP AUTOMATION**


| | |
|---|---|
| **Student** | Tuheu Tchoubi Pempeme Moussa Fahdil |
| **Programme** | B.Sc. Computer Science, final-year project |
| **Supervisor** | [Supervisor name – to be completed] |
| **Institution** | The ICT University, Cameroon |
| **Version** | 2.0 – October 2026 |

# Document Control

| Item | Description |
|---|---|
| Purpose | Define the problem, scope, requirements, system design, UML models, data model, security, testing and implementation plan for DocuMind AI. |
| Intended readers | Student developer, academic supervisor, evaluators and future maintainers. |
| Version | 2.0 (October 2026). Supersedes version 1.0, which used a 12-step roadmap and unlabelled diagrams. |
| Status | Design specification plus an honest implementation-status chapter (Chapter 15). Performance targets are provisional until measured. |
| Working name | DocuMind AI is a *proposed* name. Check availability before any commercial use. |
| Companion document | *Phase 1 – Requirements Analysis and Planning* (separate file) contains the complete Phase 1 deliverable. |
| Primary technologies | Python, FastAPI, MySQL, React, OpenCV, PyTorch, Docker; Kubernetes in a later stage. |
| Important limitation | No OCR accuracy, ERP compatibility or performance result is claimed here. Such results must come from experiments. |

**What changed in version 2.0**

- New colour theme and a consistent set of UML-style diagrams, drawn as editable TikZ code.
- The roadmap now follows the 18-phase plan; the prototype is limited to **invoices**.
- Requirements were extended (FR-21, FR-22) and linked to provisional success metrics.
- A new chapter records what is implemented, what is only written, and what is still planned.

# Abstract

Organisations receive invoices and other business documents as paper scans, PDFs and images. Manual data entry is slow, repetitive and prone to transcription errors. This project proposes DocuMind AI, an Intelligent Document Processing (IDP) and automated ERP ingestion system that accepts a document, improves its image quality, recognises its text, extracts structured fields, validates them, routes uncertain cases to a human reviewer and prepares approved records for an Enterprise Resource Planning (ERP) system.

The solution combines a trainable optical character recognition (OCR) component with explicit extraction and validation rules. Image preprocessing uses OpenCV; text recognition is developed with PyTorch using a CRNN network trained with Connectionist Temporal Classification (CTC) loss. French and English are the target languages, and the first prototype is limited to invoices. An established OCR engine serves as a baseline so that the custom model can be evaluated honestly. The backend uses FastAPI and MySQL; a React web dashboard supports upload, review and monitoring. The system starts as a modular monolith, is containerised with Docker, and is separated into services and deployed on Kubernetes only after the interfaces are stable.

This document specifies requirements, architecture, UML models, data design, security, testing and a phased plan. Measured results will replace the provisional targets once experiments have been run.

**Keywords:** intelligent document processing, OCR, information extraction, human-in-the-loop, ERP integration, FastAPI, React, MySQL, PyTorch.

# Contents

**Document Control** i

**Abstract** ii

**1 Introduction** 1

1.1 Background . . . 1
1.2 Problem Statement . . . 1
1.3 Aim . . . 1
1.4 Specific Objectives . . . 1
1.5 Research and Engineering Questions . . . 1
1.6 Expected Contribution . . . 2

**2 Scope, Users and Assumptions** 3

2.1 Initial Scope . . . 3
2.2 Out of Scope for the First Prototype . . . 3
2.3 Stakeholders and Roles . . . 3
2.4 Assumptions and Dependencies . . . 3

**3 Requirements Specification** 5

3.1 Functional Requirements . . . 5
3.2 Non-Functional Requirements . . . 6
3.3 Business Rules . . . 6
3.4 Provisional Success Metrics . . . 7

**4 System Architecture** 8

4.1 Architecture Overview . . . 8
4.2 Logical Modules . . . 8
4.3 Why Start with a Modular Monolith? . . . 9
4.4 Repository Structure . . . 9

**5 UML Modelling** 10

5.1 Use-Case Diagram . . . 10
5.2 Class Diagrams . . . 11
5.2.1 Domain model . . . 11
5.2.2 Service layer and extension points . . . 12
5.3 State Machine of a Document . . . 12

5.4 Activity Diagram . . . 14
5.5 Sequence Diagrams . . . 15

**6 OCR, Extraction and Validation Design 17**
6.1 OCR versus Document Understanding . . . 17
6.2 Processing Pipeline . . . 17
6.3 Custom Recognition Model . . . 18
6.3.1 Dataset requirements . . . 18
6.3.2 Evaluation metrics . . . 18
6.4 Field Extraction Strategy . . . 18
6.5 Validation Rules . . . 19
6.6 Confidence Policy . . . 19

**7 Data Design 20**
7.1 Principles . . . 20
7.2 Entity-Relationship Diagram . . . 20
7.3 Entity Descriptions . . . 21
7.4 Schema Excerpt . . . 21
7.5 Integrity and Indexes . . . 21

**8 API Design 22**
8.1 Conventions . . . 22
8.2 Illustrative Response . . . 22
8.3 Error Handling . . . 23

**9 Frontend Design 24**
9.1 Main Screens . . . 24
9.2 Usability Guidelines . . . 24

**10 Security, Privacy and Reliability 26**
10.1 Security Controls . . . 26
10.2 Privacy and Governance . . . 26
10.3 Reliability and Recovery . . . 26

**11 Technology Stack and Rationale 27**
11.1 Dependency Installation Strategy . . . 27

**12 Testing and Evaluation Plan 28**
12.1 Testing Levels . . . 28
12.2 Evaluation Metrics . . . 28
12.3 Experiment Design . . . 28
12.4 Acceptance Criteria for the Prototype . . . 29

**13 Implementation Roadmap** **30**

**14 Deployment and Operations** **31**

14.1 Local Development . . . . . 31

14.2 Docker Compose Stage . . . . . 31

14.3 Kubernetes Stage . . . . . 32

14.4 Observability . . . . . 32

**15 Implementation Status** **33**

15.1 Backend Foundation (Phase 3–4, partial) . . . . . 33

15.2 Not Yet Started . . . . . 33

**16 Risks and Mitigation** **34**

**17 Conclusion** **36**

**A Glossary** **37**

**B Suggested Demonstration Script** **38**

**C References and Further Reading** **39**

# List of Figures


4.1 Layered logical architecture. Solid arrows are requests; dashed arrows are data access. The job queue is introduced only in a later stage. . . . 8

5.1 Use cases and actors. Every use case requires the user to be signed in (FR-01). « include »marks behaviour that is always executed as part of another use case. . . . 10

5.2 Domain class diagram. A hollow diamond marks composition-like ownership (a processing run owns its extracted fields); plain lines are associations with multiplicities. . . . 11

5.3 Strategy pattern for OCR engines and ERP adapters. . . . 12

5.4 Document lifecycle. Initial state is the filled dot; double circles are final states. . . . 13

5.5 Swimlane activity diagram from upload to controlled ERP submission. Rejection ends the workflow without any ERP call. . . . 14

5.6 Upload to OCR result. The upload request returns before OCR finishes; the UI polls the status endpoint. . . . 15

5.7 Review, approval and ERP submission, including the success and failure branches of the integration call. . . . 16

6.1 End-to-end processing pipeline. The same segmented lines feed both the custom recogniser and the baseline engine. . . . 17

6.2 CRNN recogniser with CTC loss. Dimensions are indicative and will be tuned experimentally. 18

7.1 Logical ER diagram (PK = primary key, FK = foreign key). Final constraints and indexes are fixed when the schema is implemented. . . . 20

9.1 Wireframe of the review workspace (illustrative values from a synthetic invoice). Colour and a text label (REVIEW) are both used, so status never depends on colour alone. . . . 24

13.1 The four stages of the project. . . . 30

14.1 Docker Compose deployment. Dashed boxes are later-stage components. . . . 31

14.2 Target Kubernetes layout (later stage). . . . 32

16.1 Risk matrix (likelihood against impact). Identifiers refer to the table below; ratings are initial judgements to be revisited each phase. . . . 34

# List of Tables

# 1

# Introduction

## 1.1 Background

Business and public-sector workflows depend on information received in semi-structured or unstructured documents. A clerk opens a PDF, finds the supplier name, invoice number, date, tax amount and total, and types them into a business application. Repeated across hundreds of documents, this consumes staff time and introduces errors.

OCR converts visual text into machine-readable text, but OCR alone does not solve the problem. A practical system must also locate the relevant fields, normalise values, apply business rules, preserve the source document, track corrections and deliver approved data to the target system.

## 1.2 Problem Statement

There is a gap between incoming digital documents and structured business records. A basic OCR tool returns text without knowing which text is the invoice number, or whether the total agrees with the subtotal and tax. A useful pipeline must connect acquisition, recognition, extraction, validation, human review and controlled ERP delivery in a traceable workflow.

## 1.3 Aim

To design and implement a secure, modular document-processing system that extracts structured information from French and English invoices and prepares validated records for ERP ingestion.

## 1.4 Specific Objectives

1. Provide secure document upload and lifecycle tracking.
2. Preprocess scanned images to improve text readability.
3. Develop and evaluate a trainable OCR model for French and English text.
4. Extract key invoice fields using configurable rules and layout information.
5. Compute field-level confidence and identify records that need human review.
6. Validate fields using format checks, required-field checks, duplicate detection and arithmetic rules.
7. Provide a dashboard to review, correct, approve, reject and track documents.
8. Store metadata, extracted values, validation outcomes and audit events in MySQL.
9. Expose controlled interfaces to an ERP adapter or a sandbox ERP endpoint.
10. Package the system with Docker and document a Kubernetes path.

## 1.5 Research and Engineering Questions

- How much does image preprocessing improve recognition on the selected dataset?

- How does the custom OCR model compare with an established baseline engine?
- Which combination of confidence, validation rules and human review balances automation and correctness?
- How can documents be processed reliably without losing traceability?
- How can ERP-specific code be isolated from document-processing logic?

## 1.6 Expected Contribution

The contribution is an integrated, measurable prototype, not an unsupported claim that a custom OCR model beats existing tools. The work combines a trainable recogniser, domain-specific extraction and validation, human review, auditability and a replaceable ERP adapter. The comparison experiment will show where the custom component works well and where it needs improvement.

**Project name**

The working name DocuMind AI comes from the naming shortlist in the planning phase. It is a suggestion, not a verified trademark or domain. The name is defined in one place (`\projname`) so it can be changed everywhere at once.

# 2

# Scope, Users and Assumptions

## 2.1 Initial Scope

The first version handles **invoices** only (supplier invoices received as PDF, PNG or JPEG), in French and English. Purchase orders, receipts, forms and administrative requests are extensions for later versions. The ERP stage uses a sandbox API or a clearly labelled mock adapter until the target ERP and its interface are confirmed.

## 2.2 Out of Scope for the First Prototype

- Fully autonomous posting of every document without human review.
- Support for every document type, language, handwriting style or layout.
- A production connection to an ERP whose API, schema and authentication are not yet known.
- Claims of legal or accounting compliance without review against applicable rules.
- Kubernetes operations before the containerised application is stable and tested.

## 2.3 Stakeholders and Roles

| Role | Responsibility | Permissions / use |
|---|---|---|
| Administrator | Manage users, configuration, thresholds and integration settings. | Create or deactivate users, assign roles, view system status, configure rules. |
| Document operator | Upload and monitor incoming documents. | Upload, view status of permitted records. |
| Reviewer / accountant | Check extracted fields and resolve uncertainty. | Correct fields, add notes, approve or reject records. |
| Integration service account | Transfer approved records to the ERP. | Restricted API credential; cannot administer users or change source documents. |
| Supervisor / auditor | Assess operation and traceability. | Read authorised logs, metrics and audit history. |

## 2.4 Assumptions and Dependencies

- A labelled dataset of representative French and English invoices can be obtained lawfully or created (including synthetic invoices).
- Training and evaluation data can be separated to avoid leakage.
- The development machine runs a stable Python release; a GPU (or a cloud notebook) is optional for the prototype but useful for training.

- MySQL is available locally during early development.
- The target ERP and its integration method will be chosen before any production posting.
- Test documents contain no real sensitive data unless authorised and protected.

# 3

# Requirements Specification

Requirements use stable identifiers so that each can be traced to a design element and a test. The full Phase 1 specification, with acceptance criteria and a traceability matrix, is in the companion document.

## 3.1 Functional Requirements

| ID | Requirement | Priority |
|---|---|---|
| FR-01 | A permitted user can sign in and sign out. | Must |
| FR-02 | The system enforces role-based access to administrative, operational and review functions. | Must |
| FR-03 | A user can upload a PDF, PNG or JPEG document within configured size and type limits. | Must |
| FR-04 | The system checks file type, size and basic integrity before accepting a document. | Must |
| FR-05 | The original file is stored in protected storage, linked to a unique document identifier and a SHA-256 hash. | Must |
| FR-06 | The system creates a processing job and records its status and timestamps. | Must |
| FR-07 | The preprocessing module normalises pages and applies suitable image enhancement. | Must |
| FR-08 | The OCR component returns recognised text and confidence information where supported. | Must |
| FR-09 | The extraction component maps recognised content to the configured invoice fields. | Must |
| FR-10 | The validation component checks required fields, formats, arithmetic consistency and duplicate indicators. | Must |
| FR-11 | Low-confidence or invalid records are marked for human review. | Must |
| FR-12 | An authorised reviewer can correct fields, add a reason, and approve or reject a record. | Must |
| FR-13 | Every material correction, approval, rejection and integration attempt is recorded in an audit trail. | Must |
| FR-14 | Only approved records can be submitted to the ERP adapter. | Must |
| FR-15 | The system records ERP submission status, external reference and errors without duplicating a successful submission. | Must |
| FR-16 | Users can search and filter documents by status, date, type or identifier. | Should |
| FR-17 | Administrators can configure validation rules and review thresholds. | Should |
| FR-18 | The system exposes health endpoints and basic operational metrics. | Should |
| FR-19 | The dashboard shows processing counts and the failure and review queues. | Should |

| ID | Requirement | Priority |
|---|---|---|
| FR-20 | Approved structured data can be exported (JSON/CSV) when ERP connectivity is unavailable. | Could |
| FR-21 | A failed or rejected document can be resubmitted for reprocessing as a new processing run. | Should |
| FR-22 | The review screen shows the source page beside the extracted fields and highlights the evidence for each field. | Should |

## 3.2 Non-Functional Requirements

| ID | Category | Requirement and verification approach |
|---|---|---|
| NFR-01 | Security | Authenticate users, authorise every protected operation, keep secrets out of source code, restrict stored files. Verified by access-control tests. |
| NFR-02 | Privacy | Collect only necessary data, define retention, use synthetic or authorised test documents. |
| NFR-03 | Reliability | A failed processing step yields a visible failure state and a recoverable record, never a silently lost job. |
| NFR-04 | Traceability | Every document and processing run has stable identifiers and timestamps. |
| NFR-05 | Maintainability | Separate API, domain logic, OCR, validation, storage and ERP adapter responsibilities. |
| NFR-06 | Portability | Reproducible setup through documented steps and Docker images. |
| NFR-07 | Performance | Measure upload, preprocessing, OCR, extraction and end-to-end times on a defined test set; fix targets after a baseline run. |
| NFR-08 | Usability | Reviewers can find low-confidence fields and understand validation errors without reading logs; tested with representative users. |
| NFR-09 | Scalability | Long-running OCR is separable from interactive API requests and can later scale independently. |
| NFR-10 | Observability | Structured logs, health status, failure categories and durations, without logging passwords or full document content. |

## 3.3 Business Rules

1. A document cannot be submitted to an ERP until it has passed required validations and been approved by an authorised reviewer.
2. A low-confidence field must be visible to the reviewer; confidence is an indicator, not proof of correctness.
3. Duplicate detection flags likely duplicates for review; it never deletes a document automatically.
4. ERP requests use an idempotency key or an equivalent duplicate-prevention mechanism.
5. Corrections preserve the original extracted value and record the corrected value, actor, time and reason.
6. Failed integration attempts can be retried without creating duplicate business records.

## 3.4 Provisional Success Metrics

The targets below are **planning values**, chosen to give the experiments something concrete to aim at. They must be revised after the baseline measurement and must never be reported as results.

| Metric | Provisional target | How measured |
|---|---|---|
| Character error rate (CER) | Lower than the baseline engine on the same test set; absolute target set after baseline | Edit distance vs. verified transcription |
| Word error rate (WER) | As above | Word-level edit distance |
| Field F1 (invoice no., date, total) | $\geq 0.85$ after review-free extraction, to be revised | Exact/normalised match on held-out invoices |
| Records needing review | Reported, not targeted; aim to reduce over time | Share of records flagged |
| Processing time per page | Reported as median and 95th percentile | Timed end-to-end runs |
| Duplicate ERP submissions | 0 in all retry tests | Sandbox integration tests |

# 4

# System Architecture

## 4.1 Architecture Overview

The project is developed incrementally. The first milestone is a working API connected to MySQL. Document upload, preprocessing, OCR and review follow. Docker Compose is introduced once individual modules work, services are separated where there is a clear operational benefit, and Kubernetes is a later deployment target, not a prerequisite.

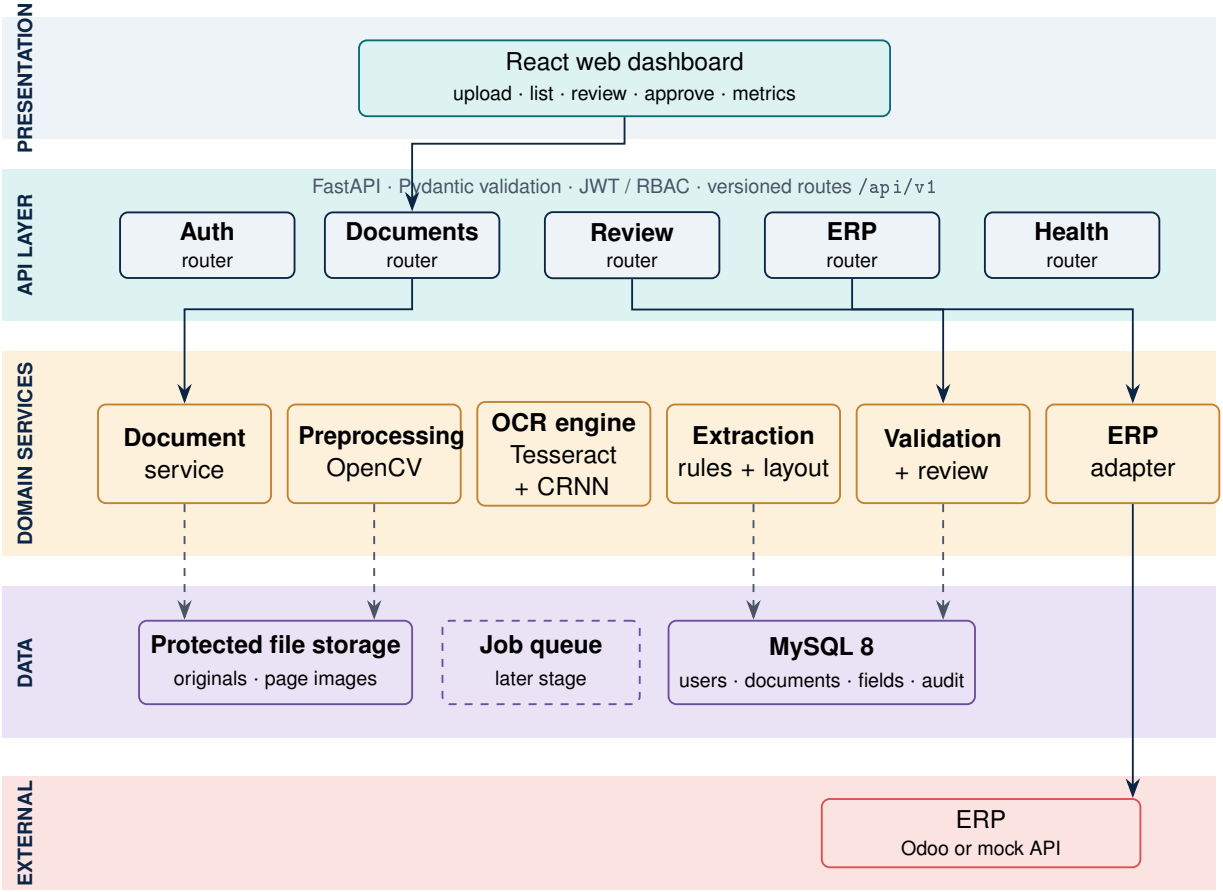



**Figure 4.1:** *Layered logical architecture. Solid arrows are requests; dashed arrows are data access. The job queue is introduced only in a later stage.*

## 4.2 Logical Modules

| Module | Responsibilities | Input → output |
|---|---|---|
| React UI | Sign-in, upload, document list, review workspace, status dashboard. | User actions → API calls and views. |
| API layer | Request validation, routing, authentication, consistent errors, API versioning. | HTTP/JSON. |
| Authentication | User identity, roles, token lifecycle, account status. | Credentials → principal. |
| Document service | Upload validation, metadata, storage reference, job lifecycle. | File → document ID and state. |
| Preprocessing / OCR | PDF rendering, image enhancement, line segmentation, recognition. | Page → text, regions, confidence. |
| Extraction | Turn OCR output into typed invoice fields and keep source evidence. | Text + layout → candidate fields. |
| Validation / review | Business rules, confidence policy, corrections, approvals. | Candidates → approved record. |
| ERP adapter | Map approved records, authenticate, submit, reconcile status. | Record → ERP response. |
| MySQL | Users, metadata, fields, workflow states, attempts, audit records. | Transactional data. |
| Protected storage | Original uploads and derived page images outside the web root. | Files. |

## 4.3 Why Start with a Modular Monolith?

Microservices add network calls, inter-service authentication, distributed failure modes and deployment effort. For a student project the sound path is to build clearly separated modules inside one backend, test their interfaces, and extract services only where needed (typically the OCR worker first, because it is slow and resource-hungry). This keeps the service-oriented design while the first milestone stays achievable. Chapter 14 shows the Docker Compose and Kubernetes stages.

## 4.4 Repository Structure

```
intelligent-document-processing/
  backend/
    app/
      main.py
      api/ # routers (health, auth, documents, review, erp)
      core/ # settings, security, logging
      database/ # engine, sessions
      models/ # SQLAlchemy models
      schemas/ # Pydantic schemas
      services/ # documents, preprocessing, ocr, extraction,
                  # validation, integration
      workers/ # background jobs (later)
    alembic/ # migrations
    tests/
    Dockerfile requirements.txt
  frontend/react_app/ # React (Node.js, Vite)
  ml/ # datasets (not committed), training, evaluation
  infrastructure/ # docker, compose, kubernetes
  docs/
  docker-compose.yml .env.example README.md
```

# 5

# UML Modelling

This chapter presents the system from several standard UML viewpoints. Each diagram answers a different question: who uses the system (use cases), what it stores (class and ER), how a document changes state, how work flows, and how components talk to each other (sequence).

## 5.1 Use-Case Diagram

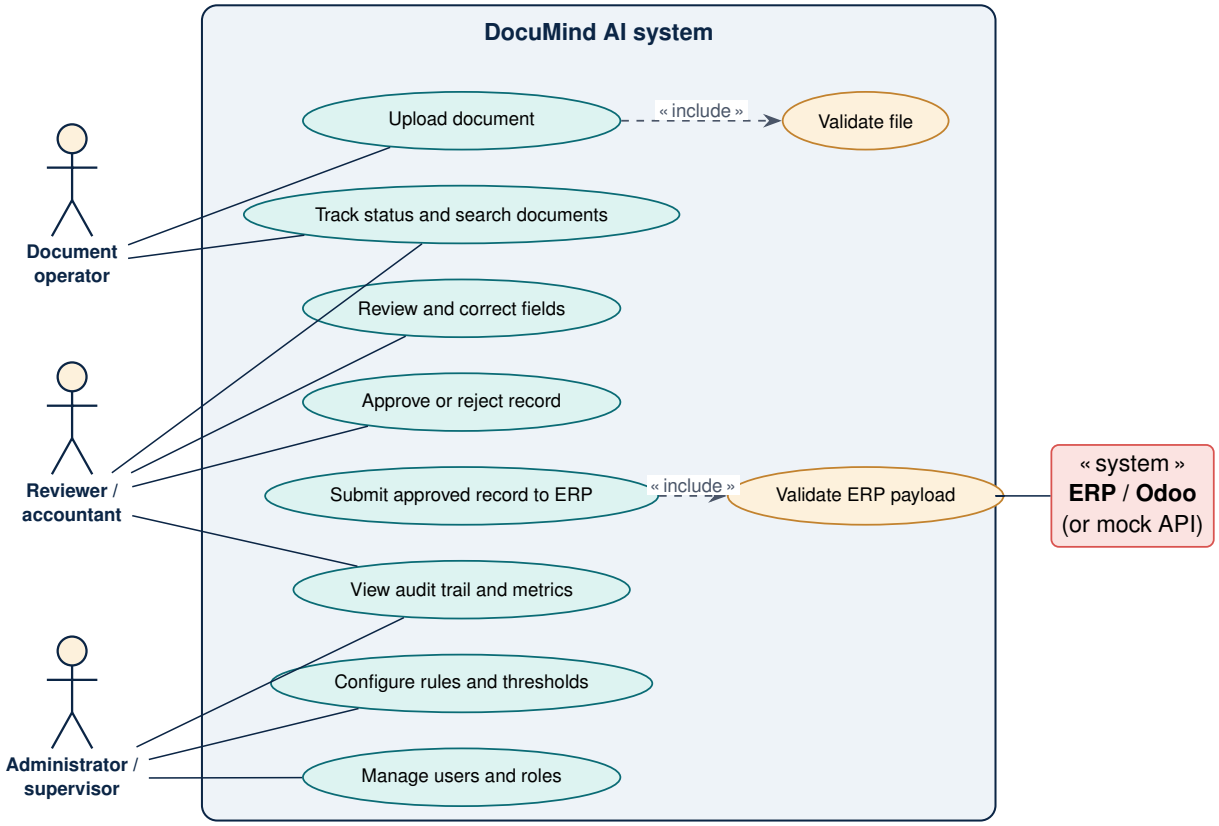



**Figure 5.1:** *Use cases and actors. Every use case requires the user to be signed in (FR-01). « include » marks behaviour that is always executed as part of another use case.*

### Use-case summary

| Use case | Actor | Outcome |
|---|---|---|
| Upload document | Operator | A valid file is stored, a document ID is returned, a job is queued. |
| Track status and search | Operator, Reviewer | Current state and history of each permitted document. |

| Use case | Actor | Outcome |
|---|---|---|
| Review and correct fields | Reviewer | Corrected values saved with reasons and audit events. |
| Approve or reject record | Reviewer | Record becomes APPROVED or REJECTED. |
| Submit approved record to ERP | ERP system | ERP reference stored; failures recorded and retryable. |
| View audit trail and metrics | Administrator, Reviewer | Traceable history and processing statistics. |
| Configure rules and thresholds | Administrator | Validation rules and review thresholds updated. |
| Manage users and roles | Administrator | Accounts created, deactivated, roles assigned. |

## 5.2 Class Diagrams

### 5.2.1 Domain model

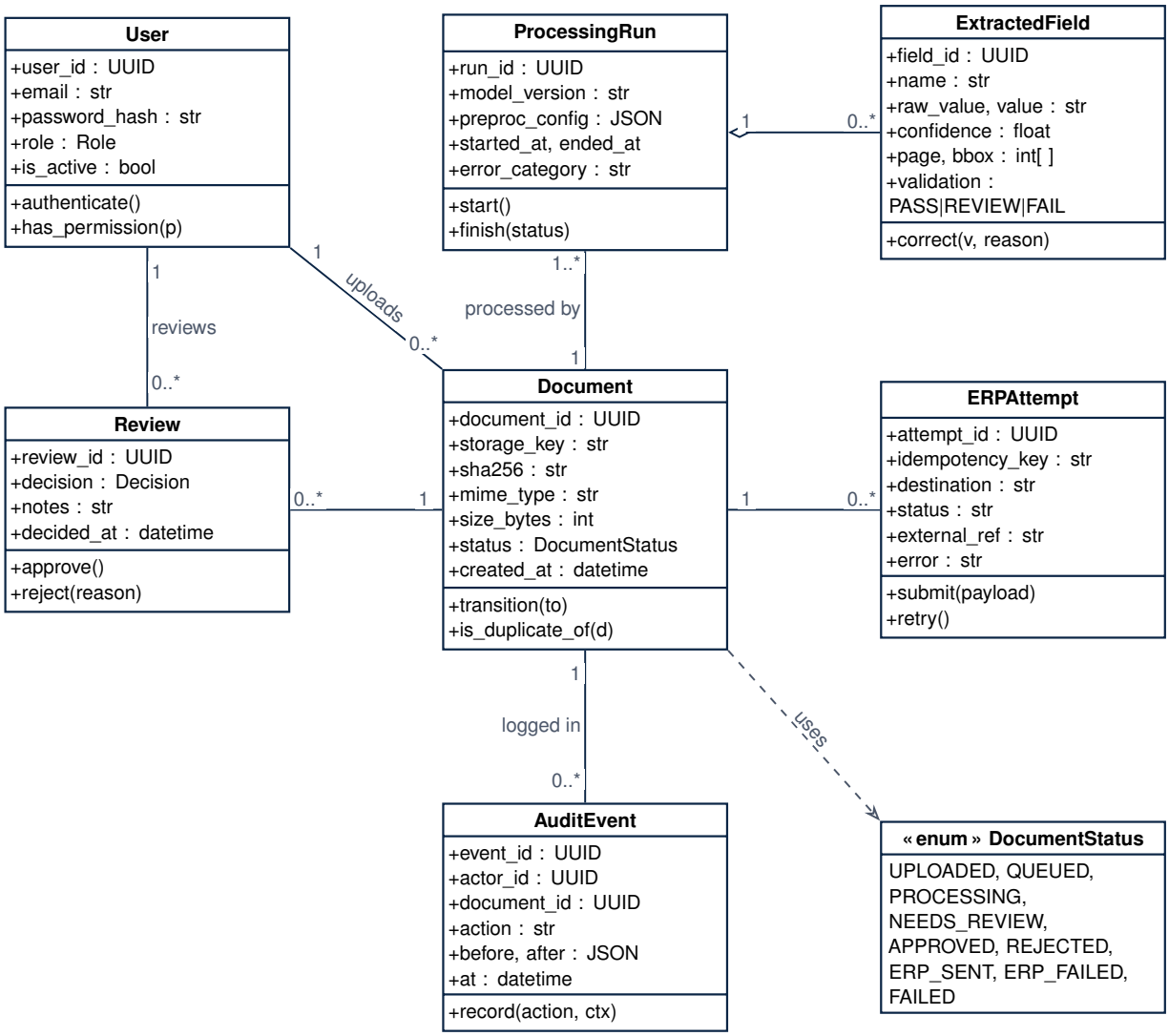
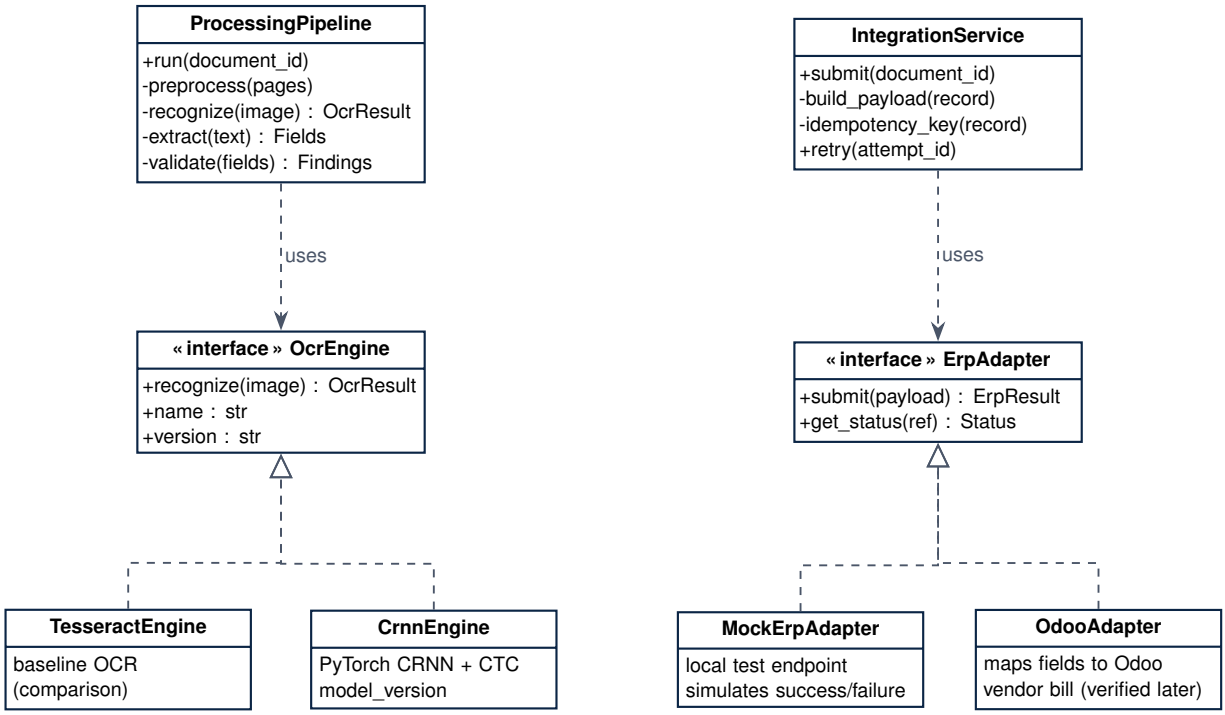



**Figure 5.2:** *Domain class diagram. A hollow diamond marks composition-like ownership (a processing run owns its extracted fields); plain lines are associations with multiplicities.*

### 5.2.2 Service layer and extension points

Two interfaces isolate the parts most likely to change. `OcrEngine` lets the pipeline use Tesseract (baseline) or the custom CRNN without modification, which is exactly what the comparison experiment needs. `ErpAdapter` keeps ERP-specific code (mock now, Odoo later) out of the document-processing logic (research question 5).




Dashed arrows with open heads show *realization*: concrete engines and adapters implement the same interface, so either can be swapped without changing the pipeline.

**Figure 5.3:** *Strategy pattern for OCR engines and ERP adapters.*

## 5.3 State Machine of a Document

A document moves through explicit states. The implementation must reject invalid transitions (for example, an unreviewed document must never jump to `ERP_SENT`).

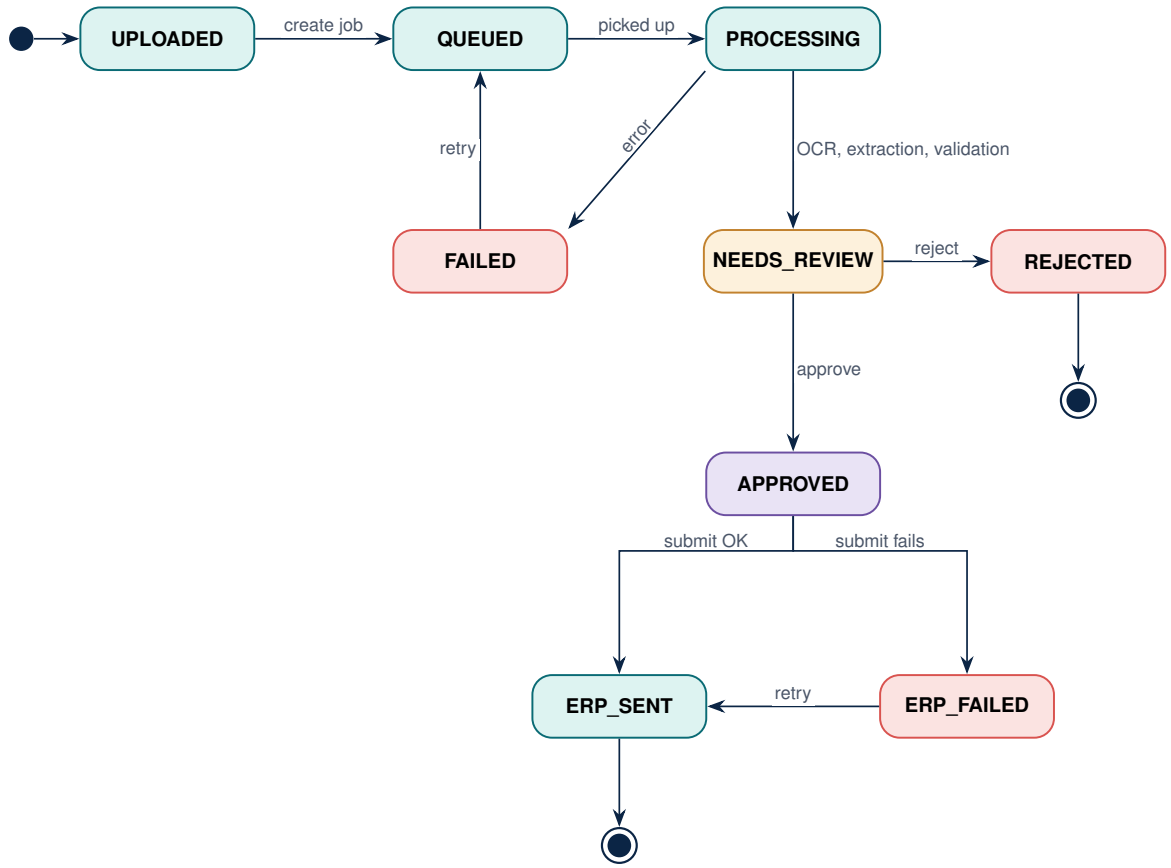



**Figure 5.4:** *Document lifecycle. Initial state is the filled dot; double circles are final states.*

| From | To | Trigger / guard |
|---|---|---|
| UPLOADED | QUEUED | Upload accepted and stored. |
| QUEUED | PROCESSING | Worker starts a processing run. |
| PROCESSING | NEEDS_REVIEW | Extraction and validation complete (human approval is mandatory in the prototype). |
| PROCESSING | FAILED | Unrecoverable step error; error category stored. |
| FAILED | QUEUED | Manual or bounded automatic retry (FR-21). |
| NEEDS_REVIEW | APPROVED / REJECTED | Authorised reviewer decision. |
| REJECTED | QUEUED | Reprocessing requested; creates a new run (FR-21). |
| APPROVED | ERP_SENT / ERP_FAILED | ERP response. |
| ERP_FAILED | ERP_SENT | Retry with the same idempotency key. |

## 5.4 Activity Diagram

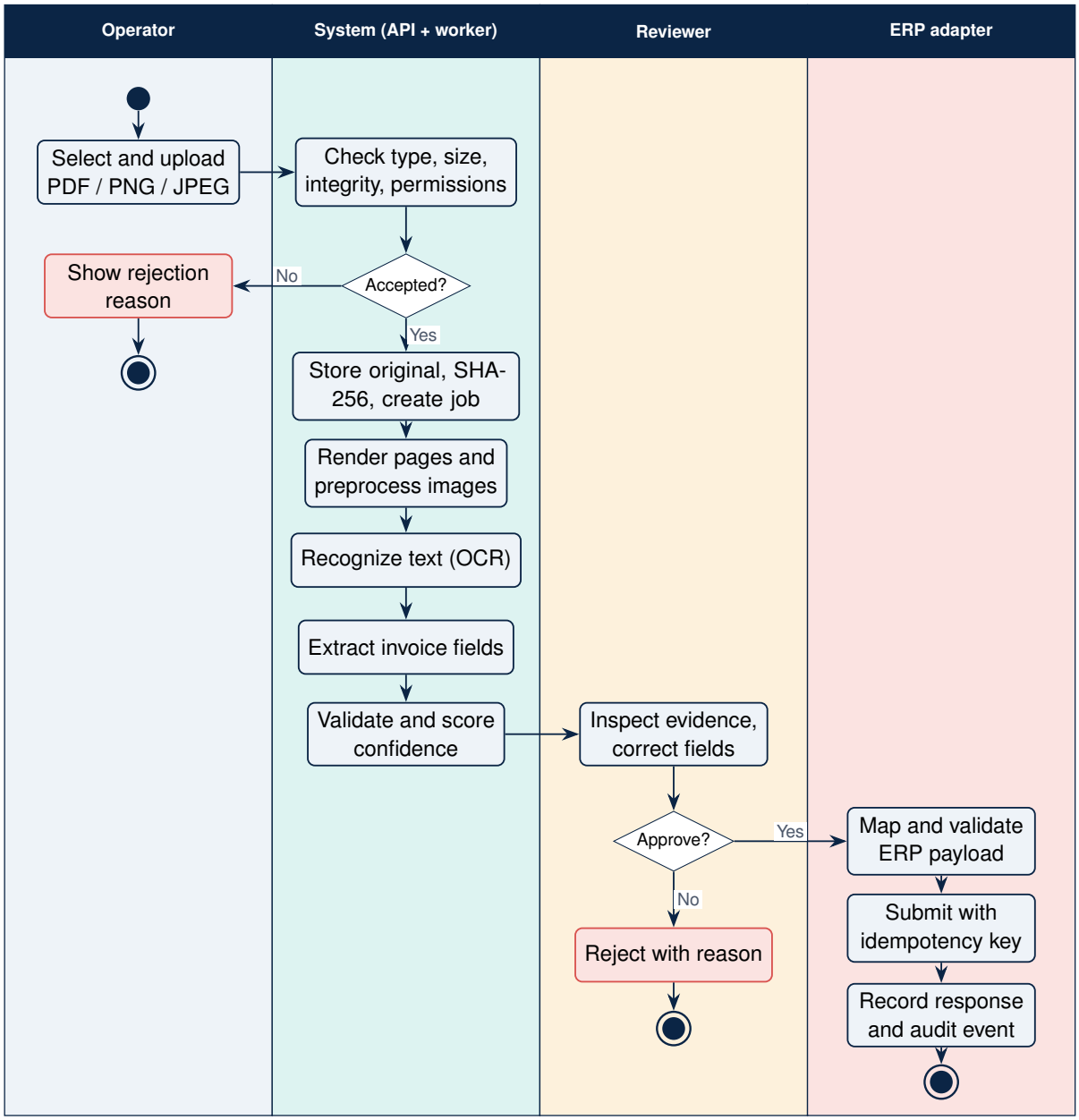



**Figure 5.5:** *Swimlane activity diagram from upload to controlled ERP submission. Rejection ends the workflow without any ERP call.*

## 5.5 Sequence Diagrams

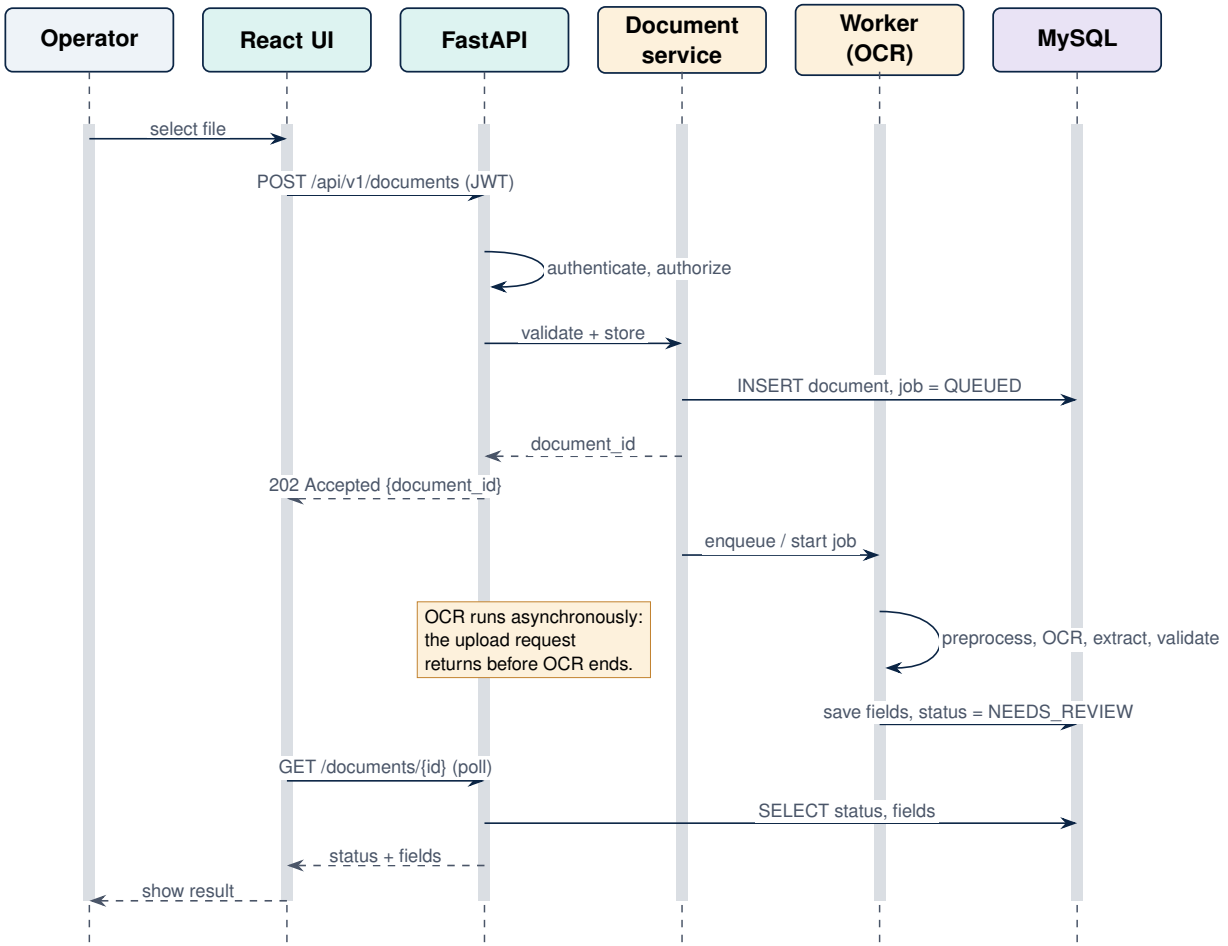



**Figure 5.6:** *Upload to OCR result. The upload request returns before OCR finishes; the UI polls the status endpoint.*

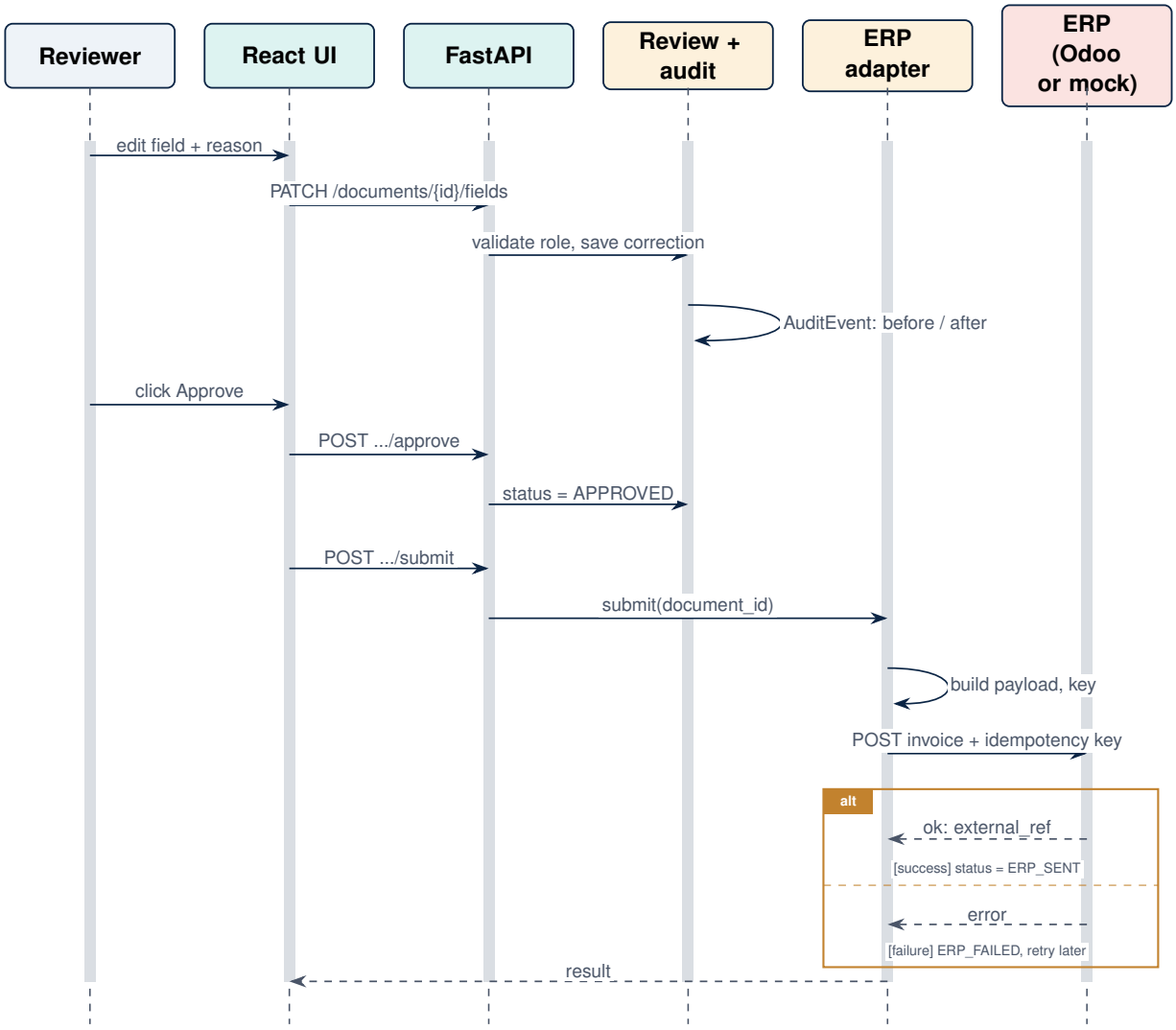



**Figure 5.7:** *Review, approval and ERP submission, including the success and failure branches of the integration call.*

# 6

# OCR, Extraction and Validation Design

## 6.1 OCR versus Document Understanding

OCR identifies characters or words in an image. Extraction decides which recognised values correspond to business fields. Validation checks whether those values are plausible and consistent. Keeping these three stages separate lets each be tested independently.

## 6.2 Processing Pipeline

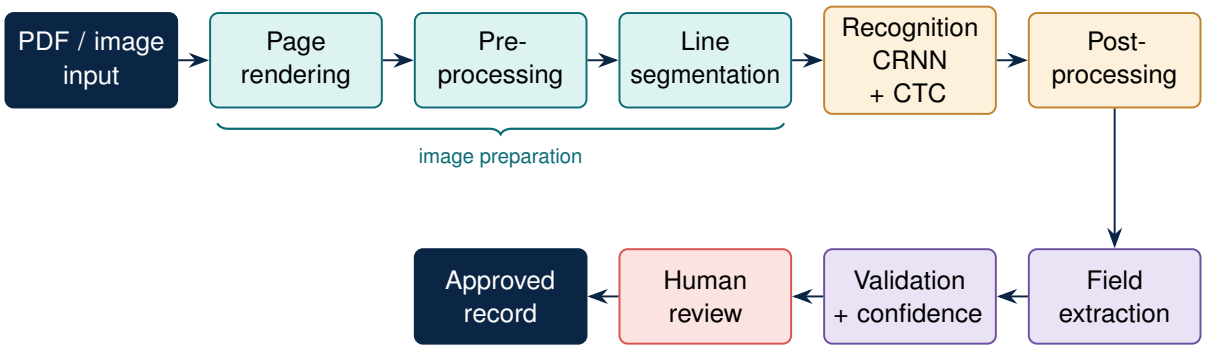
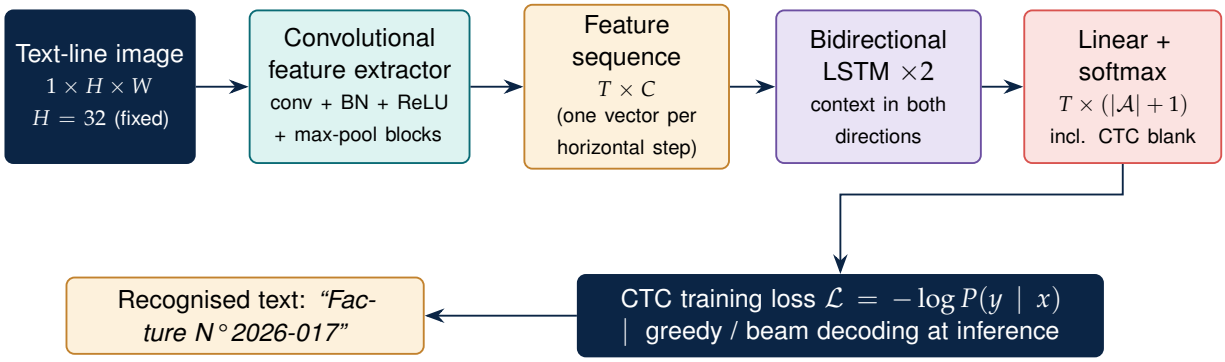



**Figure 6.1:** *End-to-end processing pipeline. The same segmented lines feed both the custom recogniser and the baseline engine.*

1. **Input inspection:** file type, page count, dimensions, rotation, basic quality.
2. **PDF rendering:** render each page at a suitable resolution while keeping the original file.
3. **Preprocessing:** grayscale, denoising, contrast normalisation, skew correction, thresholding when helpful, resizing.
4. **Text segmentation:** the first prototype uses a constrained line-based approach; complex layouts need more robust detection.
5. **Recognition:** a CRNN with CTC loss, or a justified alternative after experiments.
6. **Post-processing:** normalise whitespace and common confusions without silently altering source evidence.
7. **Field extraction:** labels, layout positions, patterns and rules propose invoice fields.
8. **Confidence and review:** combine confidence with validation outcomes and route uncertain cases to a reviewer.

## 6.3 Custom Recognition Model




$\mathcal{A}$: alphabet of French and English letters (including accented letters), digits, punctuation and business symbols. Image height and layer sizes are design choices to be tuned experimentally.

**Figure 6.2:** *CRNN recogniser with CTC loss. Dimensions are indicative and will be tuned experimentally.*

A CRNN combines a convolutional feature extractor, a bidirectional recurrent layer for sequence context and a character classifier. CTC loss allows training without character-level alignment:

$$\mathcal{L}_{\text{CTC}} = -\log P(y \mid x),$$

where $x$ is the input line image and $y$ the target text. The alphabet must cover French and English letters (including accents), digits, punctuation and business symbols.

### 6.3.1 Dataset requirements

- Representative invoice images with verified transcriptions.
- Training, validation and test sets separated *by document or source*, not by random crops of the same page.
- Clean and degraded examples: low contrast, skew, noise, different fonts, stamps and scan artefacts.
- Metadata: language, document type, image size and preprocessing settings.
- Personal or confidential content removed or protected; dataset, label and model versions tracked.

### 6.3.2 Evaluation metrics

Character error rate and word error rate are

$$\text{CER} = \frac{S + D + I}{N}, \qquad \text{WER} = \frac{S_w + D_w + I_w}{N_w},$$

with substitutions $S$, deletions $D$, insertions $I$ and reference length $N$ (characters or words). Both models are evaluated on the same held-out set. A lower value is better, and no number is written into the final report before the experiment has been run.

## 6.4 Field Extraction Strategy

For invoices the candidate fields are supplier name and tax identifier, invoice number, invoice and due dates, currency, subtotal, tax, total and purchase-order reference (Phase 1 specification, Table of invoice fields). Regular expressions handle constrained patterns; keyword anchors ("Invoice No.", "Facture N°") and page coordinates locate values; arithmetic rules cross-check amounts. Each field keeps the text span or coordinates that support it, which is what the review screen highlights (FR-22).

## 6.5 Validation Rules

| Rule | Example check | Outcome |
|---|---|---|
| Required fields | Invoice number, supplier, date and total are present. | Missing field becomes a review finding. |
| Date parsing | Date parses under configured formats and a plausible range. | Invalid or ambiguous date flagged. |
| Numeric parsing | Amounts normalised using locale conventions (space or dot thousands). | Unparseable amount flagged. |
| Arithmetic | Subtotal + tax $\approx$ total within a configured tolerance. | Mismatch flagged, never auto-corrected. |
| Currency | Symbols and codes agree across fields. | Inconsistency flagged. |
| Duplicate indicator | Supplier, invoice number, date, amount, optional file hash. | Likely duplicate flagged. |
| ERP schema | Approved values satisfy the ERP payload schema. | Invalid payload blocked before sending. |

## 6.6 Confidence Policy

Confidence is a decision signal, not a guarantee. Thresholds are set empirically on validation data, never on the test set. A possible policy: high-confidence fields without rule violations are shown as accepted; medium-confidence fields are highlighted; low-confidence fields and critical validation failures require human review. In the prototype, human approval before ERP submission is mandatory.

# 7

# Data Design

## 7.1 Principles

MySQL stores structured application data, not public copies of uploaded files. Originals live in protected storage (a private directory in development, private object storage in deployment). The database keeps the storage key, checksum, file metadata, status, extracted fields and audit data. File contents are never placed in a public web directory and internal paths are never exposed by the API.

## 7.2 Entity-Relationship Diagram

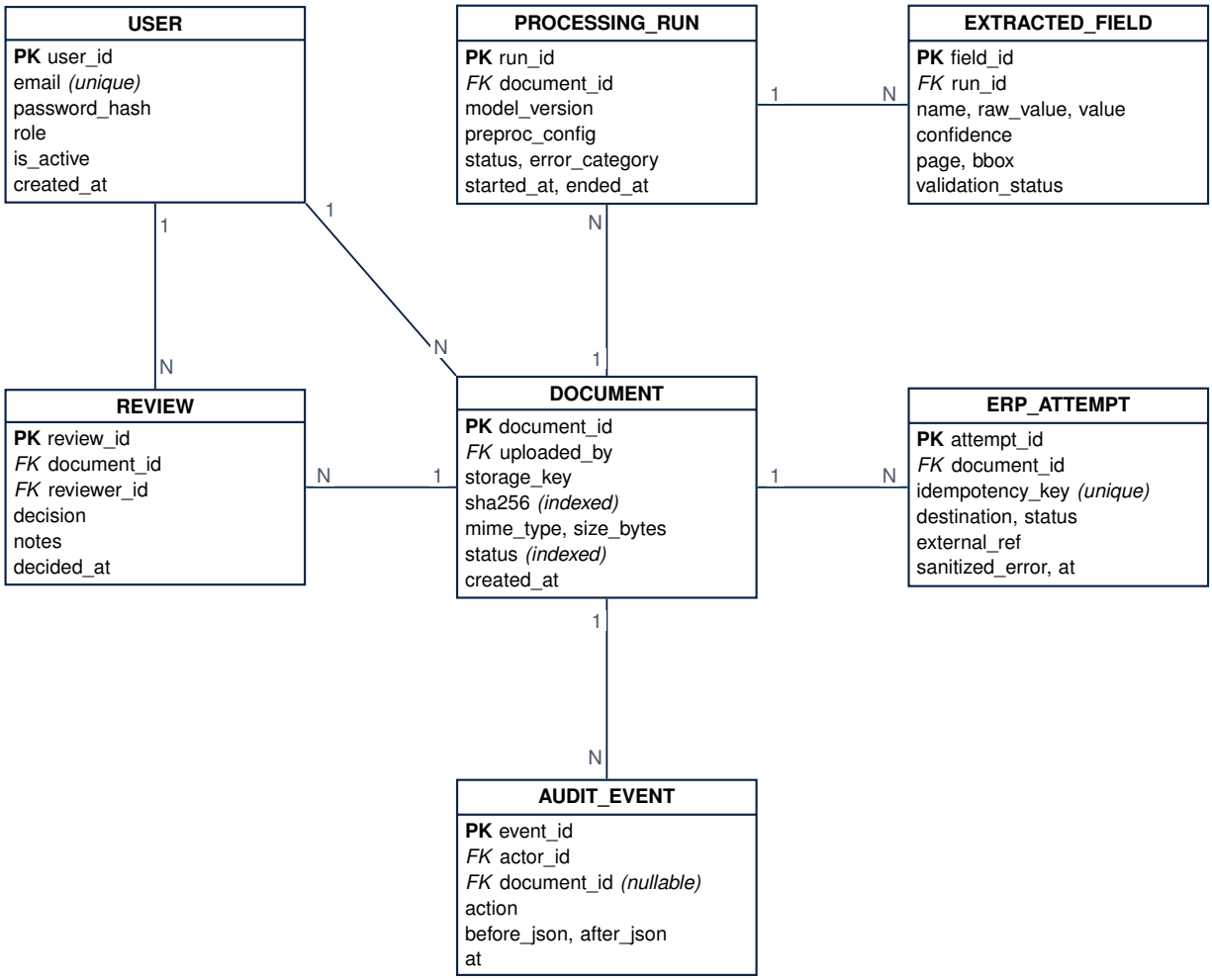



**Figure 7.1:** *Logical ER diagram (PK = primary key, FK = foreign key). Final constraints and indexes are fixed when the schema is implemented.*

## 7.3 Entity Descriptions

| Entity | Purpose and key fields |
|---|---|
| User | Identity and role. A password hash is stored only if local passwords are used; never plaintext. |
| Document | One uploaded source document: storage key, SHA-256, MIME type, size, uploader, timestamps and current workflow status. |
| ProcessingRun | One attempt to process a document: preprocessing configuration, OCR model version, start/end time, status and error category. |
| ExtractedField | A candidate field of a run: name, raw and normalised value, confidence, page/region evidence and validation status. |
| Review | Human decision with reviewer, notes, timestamp and references to before/after values. |
| ERPAttempt | Each outbound attempt: idempotency key, destination, sanitised request/response metadata, external reference, status. |
| AuditEvent | Append-only record of upload, correction, approval, rejection, configuration changes and ERP submission. |

## 7.4 Schema Excerpt

The following MySQL excerpt illustrates intended constraints. It is a design sketch to be implemented with Alembic migrations, not a deployed schema.

```
CREATE TABLE documents (
  document_id CHAR(36) PRIMARY KEY,
  uploaded_by CHAR(36) NOT NULL,
  storage_key VARCHAR(255) NOT NULL,
  sha256 CHAR(64) NOT NULL,
  mime_type VARCHAR(100) NOT NULL,
  size_bytes BIGINT NOT NULL,
  status VARCHAR(20) NOT NULL,
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (uploaded_by) REFERENCES users(user_id),
  INDEX idx_documents_status (status),
  INDEX idx_documents_sha256 (sha256)
);
CREATE TABLE erp_attempts (
  attempt_id CHAR(36) PRIMARY KEY,
  document_id CHAR(36) NOT NULL,
  idempotency_key VARCHAR(80) NOT NULL UNIQUE,
  status VARCHAR(20) NOT NULL,
  external_ref VARCHAR(100),
  FOREIGN KEY (document_id) REFERENCES documents(document_id)
);
```

## 7.5 Integrity and Indexes

Use primary and foreign keys, non-null constraints for required values, a unique constraint on user e-mail and on the idempotency key, and indexes on document status, creation date, file hash and processing status. Duplicate-invoice detection is a *business rule* rather than only a unique constraint, because extracted data may be incomplete. Schema changes go through migrations.

# 8

# API Design

## 8.1 Conventions

REST-style endpoints with JSON, multipart form data for uploads, a consistent error format and versioned routes under `/api/v1`. Input is validated with Pydantic schemas. A document identifier is returned immediately and OCR runs asynchronously.

| Method | Endpoint | Purpose |
|---|---|---|
| GET | `/api/v1/health` | Liveness; implemented in the current backend. |
| GET | `/api/v1/health/db` | Readiness (database reachable); implemented. |
| POST | `/api/v1/auth/login` | Authenticate and return a token. |
| POST | `/api/v1/documents` | Upload a document and create a job. |
| GET | `/api/v1/documents` | List and filter permitted documents. |
| GET | `/api/v1/documents/{id}` | Metadata, status and processing summary. |
| GET | `/api/v1/documents/{id}/result` | Extracted fields, confidence, findings. |
| GET | `/api/v1/documents/{id}/file` | Source file, after authorisation. |
| PATCH | `/api/v1/documents/{id}/fields` | Save reviewer corrections with audit data. |
| POST | `/api/v1/documents/{id}/approve` | Approve after authorisation and checks. |
| POST | `/api/v1/documents/{id}/reject` | Reject with a reason. |
| POST | `/api/v1/documents/{id}/submit` | Submit an approved record to the ERP adapter. |
| POST | `/api/v1/documents/{id}/reprocess` | Start a new processing run (FR-21). |

## 8.2 Illustrative Response

```
{
  "document_id": "doc_example_id",
  "status": "NEEDS_REVIEW",
  "document_type": "invoice",
  "fields": [
    {"name": "invoice_number", "value": "INV-2026-001",
     "confidence": 0.91, "validation": "PASS"},
    {"name": "total_amount", "value": "125000",
     "confidence": 0.68, "validation": "REVIEW"}
  ],
  "review_required": true
}
```

This is an illustrative contract, not output of an implemented endpoint.

## 8.3 Error Handling

Use 400 for malformed requests, 401 unauthenticated, 403 insufficient permissions, 404 missing or inaccessible resources, 409 conflicting state transitions, 413 oversized uploads, 415 unsupported media types and 500 for unexpected errors. Responses must not expose stack traces, credentials, file-system paths or sensitive internals.

# 9

# Frontend Design

## 9.1 Main Screens

1. **Sign-in:** authentication, validation feedback, account status.
2. **Dashboard:** totals by status, recent activity, failures and review queue.
3. **Upload:** file selection or drag-and-drop, accepted formats, size limits, progress.
4. **Document list:** search, filters, status, date, pagination.
5. **Review workspace:** source preview beside editable fields, confidence indicators, evidence highlighting, correction reason, approve and reject.
6. **Integration history:** submission status, attempts, sanitised errors, external reference.
7. **Administration:** users and roles, validation configuration, thresholds.

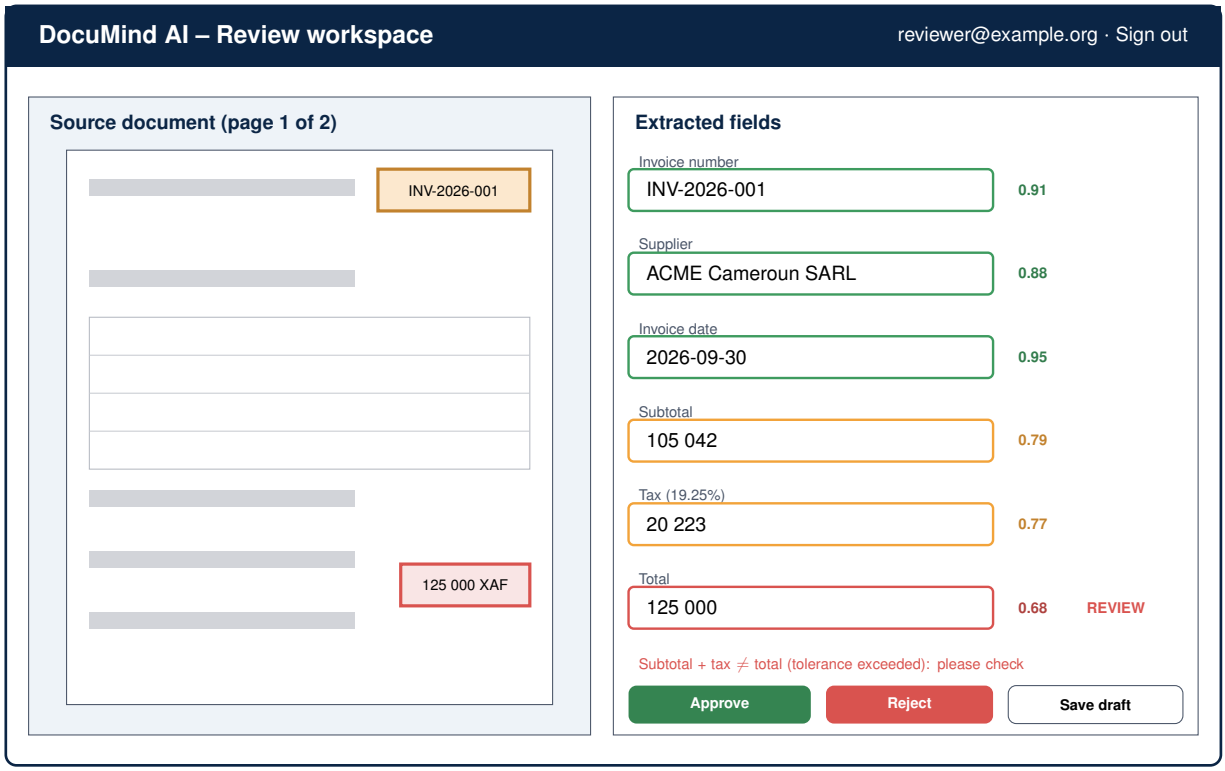



**Figure 9.1:** *Wireframe of the review workspace (illustrative values from a synthetic invoice). Colour and a text label (REVIEW) are both used, so status never depends on colour alone.*

## 9.2 Usability Guidelines

Use clear status labels, not colour alone. Keep low-confidence values prominent but let users inspect the source evidence. Show why a field failed validation. Confirm before approval or rejection and prevent

double-clicks from creating duplicate submissions. Provide loading, empty, success and error states. The React app never contains database credentials or ERP secrets.

# 10

# Security, Privacy and Reliability

## 10.1 Security Controls

- Enforce authentication and role-based authorisation in the backend, not only in the UI.
- Hash passwords with a modern algorithm if local passwords are used.
- Keep API keys, database passwords, signing keys and ERP credentials in environment-specific secret storage; commit only placeholders in `.env.example`.
- Restrict extensions, MIME types, size and PDF page count; do not trust client-supplied MIME types alone.
- Store files outside the public directory and serve them only after an authorisation check.
- Use parameterised database access through the ORM and validate all input.
- Apply rate limits and upload quotas when exposed beyond a trusted network.
- Use HTTPS when deployed and restrict CORS to known origins.
- Redact tokens, passwords and sensitive values from logs.
- Back up the database and file store together and test restoration.

## 10.2 Privacy and Governance

Documents may contain personal, financial or confidential information. Define who may access them, why they are kept, for how long and how deletion works. Use synthetic or anonymised samples for demonstrations. Do not send documents to external OCR or AI services unless the data-handling implications and authorisation are understood.

## 10.3 Reliability and Recovery

Each processing step has a status and an error category. Retries are bounded and distinguish transient from permanent errors. Job state is persisted so a restart does not silently lose work. ERP submission is idempotent where possible. Database transactions protect related metadata changes, and file writes include cleanup or reconciliation for partial failures.

# 11

# Technology Stack and Rationale

| Technology | Role | Rationale |
|---|---|---|
| Python 3.13 (recommended) | Backend and ML language | Widely used for APIs, image processing and training. Use a release supported by all dependencies; avoid pre-release interpreters. |
| FastAPI + Uvicorn | HTTP API and ASGI server | Type-aware validation and automatic OpenAPI documentation. |
| Pydantic | Schemas and settings | Validated, consistent data contracts. |
| SQLAlchemy + Alembic | ORM and migrations | Organised database access and reproducible schema changes. |
| MySQL 8 | Relational database | Metadata, fields, workflow state, users and audit records. |
| OpenCV | Image preprocessing | Denoising, thresholding and geometric correction. |
| PyTorch | Custom OCR model | Training, evaluation and inference. |
| Tesseract | Baseline OCR | Established engine for a fair comparison. |
| React (Node.js, Vite) | User interface | Component-based UI library; Node.js provides the development tooling and build. |
| Docker / Compose | Packaging and local orchestration | Reproducible environments and multi-container tests. |
| RabbitMQ + worker (later) | Background jobs | Decouples slow OCR from API requests. |
| Kubernetes (later) | Orchestration | Deploy and scale after the containerised system is stable. |

## 11.1 Dependency Installation Strategy

Do not install every dependency at once. Start from a supported Python version and a small API environment, verify the health endpoint, then add database, upload, preprocessing, OCR and UI in separate milestones. Pin tested versions. If installation fails, diagnose the Python version and the *first* real error before adding compilers or unrelated packages. (A real example from this project: installing the optional "standard" extras of FastAPI on a very new Python tried to compile PyYAML; installing `fastapi` and `uvicorn` without extras on Python 3.13 avoids that.)

# 12

# Testing and Evaluation Plan

## 12.1 Testing Levels

| Level | Examples | Evidence |
|---|---|---|
| Unit | File validation, normalisation, date/amount parsing, arithmetic rules, confidence policy. | Automated report with expected vs actual. |
| API | Authentication, upload, list, result, approval, forbidden access. | Status codes and schema checks. |
| Database | Foreign keys, constraints, migrations, rollback. | Test database results. |
| OCR evaluation | CER/WER for custom model and baseline on the same test set. | Dataset description, versions, tables. |
| End-to-end | Upload, process, review, approve, submit to sandbox. | Traceable run with state transitions. |
| Security | Wrong-role access, unsafe files, oversized uploads, secret leakage. | Pass/fail checklist and findings. |
| Recovery | OCR failure, database interruption, retry, duplicate ERP response. | Consistent status; no duplicate records. |
| Usability | Reviewer completes correction and approval tasks. | Observations and improvements. |

## 12.2 Evaluation Metrics

- **OCR:** CER, WER and inference time per page.
- **Extraction:** precision, recall and F1 per field with a clear matching rule.
- **Validation:** true errors detected, false alarms and misses on labelled cases.
- **Workflow:** share completed without correction, share sent to review, rejection rate, failures.
- **System:** median and 95th-percentile time, API error rate, retry success, resource use under a stated workload.
- **ERP:** valid payload rate, outcomes, duplicate-prevention tests.

## 12.3 Experiment Design

Create a test set with verified transcriptions and field values. Run the custom model and the baseline on exactly the same held-out examples. Report dataset size, language mix, document types, preprocessing, model versions and hardware. Do not tune thresholds on the test set. Report field-level metrics, not a single accuracy that hides weak fields, and discuss failure cases such as skewed scans, stamps, low resolution, unusual layouts and ambiguous date or number formats.

## 12.4 Acceptance Criteria for the Prototype

1. A permitted user uploads a supported invoice and receives a stable document ID.
2. The original is stored privately and a persistent state is shown.
3. A supported invoice passes preprocessing, OCR, extraction and validation.
4. A reviewer inspects the source, corrects a field and approves or rejects.
5. Only an approved record reaches the ERP adapter.
6. A sandbox or mock integration demonstrates success, failure and safe retry.
7. OCR and extraction results from a held-out set are reported with a baseline comparison.
8. Automated tests cover core rules and authorisation boundaries.
9. A new developer can follow the documented setup from a clean environment.

# 13

# Implementation Roadmap

The project follows 18 phases grouped in four stages. The key principle is to make *one complete workflow* work first – upload, preprocess, recognise, extract, validate, review, send to a test ERP – and only then scale the architecture.

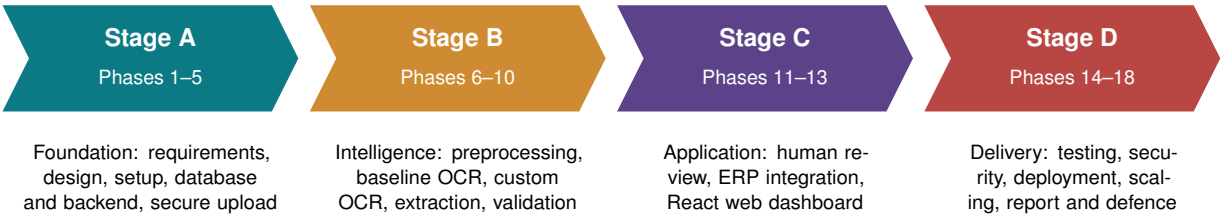



**Figure 13.1:** *The four stages of the project.*

| No. | Phase | Main deliverable | Status |
|---|---|---|---|
| 1 | Requirements and planning | Requirements specification (companion document) | Partial |
| 2 | System design and architecture | Architecture, UML, ER, API specification (this document) | Partial |
| 3 | Environment setup | Working development environment, repository | Partial |
| 4 | Database and backend foundation | Backend connected to MySQL, models, authentication | Partial |
| 5 | Document upload and storage | Secure upload, hashing, listing, retrieval | Planned |
| 6 | Image preprocessing | Pipeline preparing pages for OCR | Planned |
| 7 | Baseline OCR integration | OCR pipeline and baseline results | Planned |
| 8 | Custom OCR model | Trained model with documented evaluation | Planned |
| 9 | Information extraction | Structured invoice fields | Planned |
| 10 | Validation and confidence | Accepted / rejected / review-required records | Planned |
| 11 | Human review and correction | Review dashboard and correction workflow | Planned |
| 12 | ERP integration | Adapter with mock or sandbox ERP | Planned |
| 13 | React web dashboard | Functional web interface | Planned |
| 14 | Testing and evaluation | Test reports and evaluation tables | Planned |
| 15 | Security and reliability | Hardened, recoverable application | Planned |
| 16 | Dockerisation and deployment | Deployed application | Planned |
| 17 | Microservices and scalability | Modular deployment and scaling strategy | Planned |
| 18 | Documentation and defence | Report, slides, demonstration | Planned |

Phases 1–4 are marked “Partial” because they are only partly complete; see Chapter 15 for exact evidence.

# 14

# Deployment and Operations

## 14.1 Local Development

Initially run the API locally and keep uploaded files in a private data directory excluded from Git. Keep credentials in a local `.env` that Git ignores; `.env.example` contains placeholders only. Use a separate development database and never experiment with destructive resets on important data.

## 14.2 Docker Compose Stage

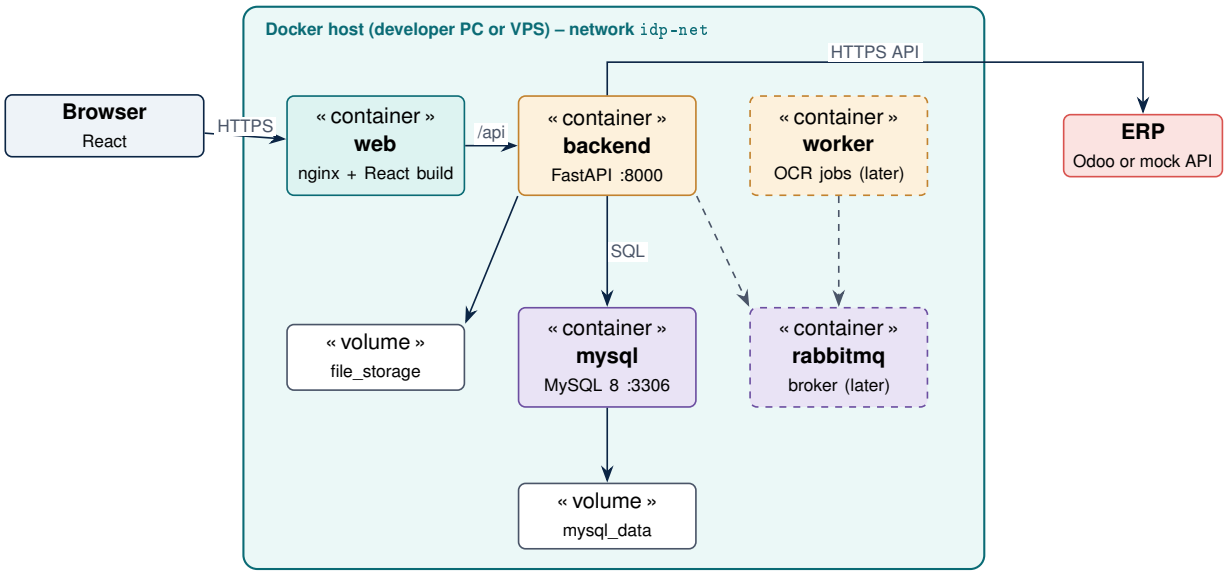



**Figure 14.1:** *Docker Compose deployment. Dashed boxes are later-stage components.*

Compose runs the API, MySQL and later the worker in separate containers on a private network with named volumes, health checks and non-root containers where practical. MySQL is not exposed to the internet and secrets are never baked into images.

## 14.3 Kubernetes Stage

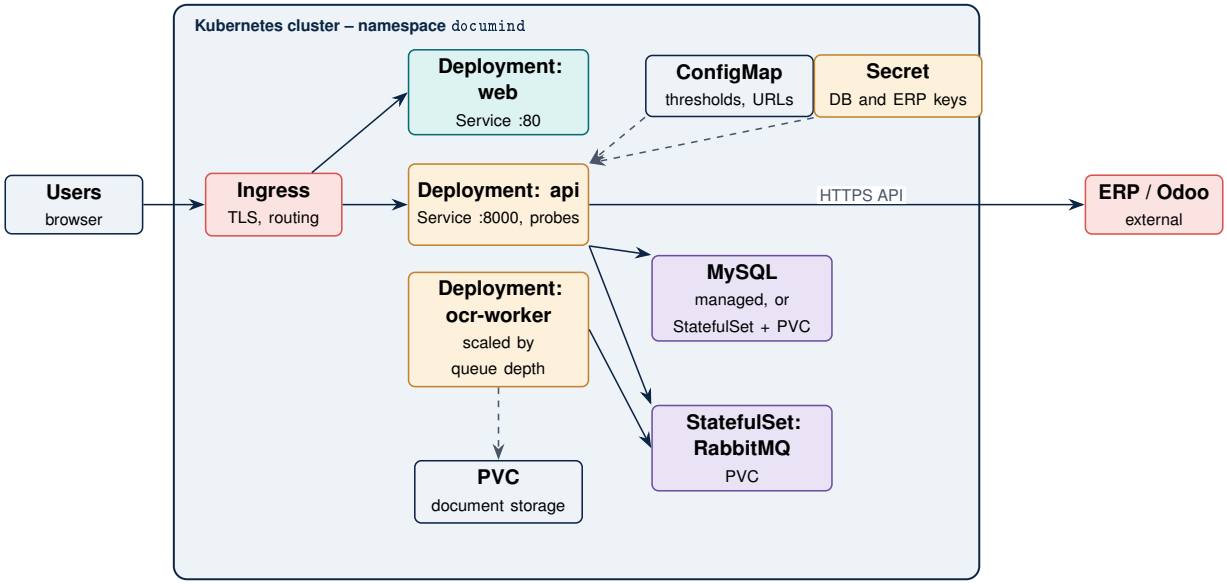



**Figure 14.2:** *Target Kubernetes layout (later stage).*

Kubernetes is attempted only after the Compose tests pass. Likely resources are Deployments, Services, ConfigMaps, Secrets, persistent volumes or object storage, readiness and liveness probes, resource limits and an Ingress with TLS. A stateful database may stay outside the cluster for a student demonstration. Secrets are never committed as plain text.

## 14.4 Observability

Use structured logs with correlation IDs (document ID, run ID). Monitor queue depth, processing duration, OCR failure categories, review backlog, ERP failure rate and database availability. Do not log full document contents by default. The health endpoint reports availability without exposing credentials.

# 15

# Implementation Status

This chapter separates what exists from what is only designed, so that the report never overstates progress.

## 15.1 Backend Foundation (Phase 3–4, partial)

| Item | Status | Evidence / note |
|---|---|---|
| FastAPI application with settings from environment | Done | Package `idp-phase1.zip`, `backend/app`. |
| Liveness endpoint `/api/v1/health` | Done | Tested; server started and endpoint answered. |
| Readiness endpoint `/api/v1/health/db` | Done | Returns 503 when MySQL is unreachable (tested); a successful MySQL connection has *not* been verified yet. |
| Automated tests | Done | 4 tests passed in the build environment (Python 3.13). |
| Dockerfile and Docker Compose with MySQL 8 | Partial | Written but *not* built or run yet; must be tested on the student's machine. |
| Run on the student's Windows machine | Planned | Pending; earlier install attempts failed on Python 3.15 – use Python 3.13. |
| SQLAlchemy models and Alembic migrations | Planned | Next step (Phase 4). |
| Authentication and roles | Planned | Phase 4. |

## 15.2 Not Yet Started

Document upload, preprocessing, baseline OCR, **custom OCR model (no model has been trained)**, extraction, validation, review workflow, ERP adapter, React dashboard, security hardening, Kubernetes. The OCR model requires a labelled dataset that does not exist yet; its training and evaluation are scheduled in Phase 8.

# 16

# Risks and Mitigation




**Figure 16.1:** *Risk matrix (likelihood against impact). Identifiers refer to the table below; ratings are initial judgements to be revisited each phase.*

| ID | Risk | Impact | Mitigation |
|---|---|---|---|
| R1 | Insufficient labelled data | Custom OCR cannot be evaluated reliably. | Limit to invoices, create synthetic and carefully labelled data, document limitations, compare with a baseline. |
| R2 | Custom OCR under-performs | Low field quality, much manual review. | Baseline comparison, error analysis, better data and preprocessing, replaceable OCR interface. |
| R3 | Python/package incompatibility | Installation and development delays. | Python 3.13, pinned versions, incremental installs. |
| R4 | Unclear ERP API or no Odoo access | Integration cannot be completed early. | Generic adapter interface, mock/sandbox, gather ERP requirements early. |
| R5 | Sensitive documents exposed | Privacy and reputational harm. | Synthetic data, private storage, authorisation, minimal logs, retention rules. |
| R6 | Duplicate ERP records | Incorrect business data. | Idempotency keys, stored attempts, retry tests. |
| R7 | Scope grows too large | Core workflow unfinished. | Invoices only, limited field set, one baseline, one custom experiment, mock ERP. |

| ID | Risk | Impact | Mitigation |
|---|---|---|---|
| R8 | Microservice overhead | More configuration and distributed failures. | Modular monolith first; split later. |
| R9 | OCR jobs crash | Documents get stuck. | Persisted status, bounded retries, manual retry. |
| R10 | Training hardware or time insufficient | Model cannot be trained or tuned. | Small model, cloud notebook or GPU access, reduce dataset scope. |

# 17

# Conclusion

DocuMind AI proposes a complete workflow that turns incoming invoices into reviewed, validated, ERP-ready records. Its value depends on more than OCR: reliable file handling, extraction, validation, human review, secure storage, audit history and careful integration behaviour all matter. A custom OCR model is a meaningful research component only if it is trained on an appropriate dataset and compared fairly with a baseline.

The recommended order is incremental: API and database, secure upload, preprocessing and baseline OCR, custom OCR and extraction, review interface and ERP adapter. Containerisation, services, queues and Kubernetes come when the simpler system works and the added complexity is justified. Before submission, complete the supervisor details, choose the ERP or sandbox, add dataset documentation and measured results, and distinguish finished features from planned ones.

# A

# Glossary

| Term | Meaning |
|---|---|
| API | Application Programming Interface: a defined way for software components to communicate. |
| CER / WER | Character / word error rate: edit errors divided by reference length. |
| CRNN | Convolutional recurrent neural network used for text recognition. |
| CTC | Connectionist Temporal Classification: loss and decoding for sequence recognition without explicit alignment. |
| ERP | Enterprise Resource Planning system. |
| Human-in-the-loop | Workflow in which a person reviews or corrects uncertain automated results. |
| IDP | Intelligent Document Processing: capture, recognise, extract, validate and route document information. |
| Idempotency | Property that makes repeated identical requests produce no duplicate effect. |
| OCR | Optical Character Recognition. |
| RBAC | Role-Based Access Control. |
| UML | Unified Modeling Language. |

# B

# Suggested Demonstration Script

1. Sign in as a document operator.
2. Upload a synthetic invoice; show its document ID and queued status.
3. Show the processing result and extracted fields.
4. Open a low-confidence field and compare it with the source.
5. Correct the field and enter a review note.
6. Approve the record and show the audit event.
7. Submit the approved record to the sandbox ERP adapter; show the external reference.
8. Submit again and show duplicate prevention.
9. Show the evaluation summary: custom OCR vs. baseline.

# C

# References and Further Reading

Consult the official documentation and record exact versions and access dates in the final report.

- FastAPI: https://fastapi.tiangolo.com/
- React: https://react.dev/
- MySQL: https://dev.mysql.com/doc/
- OpenCV: https://docs.opencv.org/
- PyTorch: https://pytorch.org/docs/
- Docker: https://docs.docker.com/
- Kubernetes: https://kubernetes.io/docs/
- SQLAlchemy: https://docs.sqlalchemy.org/
- Alembic: https://alembic.sqlalchemy.org/

**Before submission**

Complete the supervisor name, confirm your institution's formatting rules, verify each diagram against the final design, and replace planned or illustrative statements with implementation evidence.